

Twitch: Learning Abstractions for Equational Theorem Proving

Guy Axelrod Moa Johansson Nicholas Smallbone

Chalmers University of Technology and University of Gothenburg

AITP 2026

Orient equations into rewrite rules, e.g. group axioms:

$$e \cdot x \rightarrow x, \quad x^{-1} \cdot x \rightarrow e, \quad (x \cdot y) \cdot z \rightarrow x \cdot (y \cdot z).$$

¹L. Wos, *The resonance strategy*, Computers & Mathematics with Applications, 1995.

Orient equations into rewrite rules, e.g. group axioms:

$$e \cdot x \rightarrow x, \quad x^{-1} \cdot x \rightarrow e, \quad (x \cdot y) \cdot z \rightarrow x \cdot (y \cdot z).$$

Overlapping rules rewrite one term in different ways; the endpoints form a **critical pair**:

$$y \longleftarrow e \cdot y \longleftarrow (x^{-1} \cdot x) \cdot y \longrightarrow x^{-1} \cdot (x \cdot y) \quad \rightsquigarrow \quad x^{-1} \cdot (x \cdot y) \rightarrow y.$$

True, but not provable by rewriting alone — so it is oriented and added as a **new rule**.
Completion repeats: generate critical pairs, **select** one, add the rule.

¹L. Wos, *The resonance strategy*, Computers & Mathematics with Applications, 1995.

Orient equations into rewrite rules, e.g. group axioms:

$$e \cdot x \rightarrow x, \quad x^{-1} \cdot x \rightarrow e, \quad (x \cdot y) \cdot z \rightarrow x \cdot (y \cdot z).$$

Overlapping rules rewrite one term in different ways; the endpoints form a **critical pair**:

$$y \longleftarrow e \cdot y \longleftarrow (x^{-1} \cdot x) \cdot y \longrightarrow x^{-1} \cdot (x \cdot y) \quad \rightsquigarrow \quad x^{-1} \cdot (x \cdot y) \rightarrow y.$$

True, but not provable by rewriting alone — so it is oriented and added as a **new rule**.
Completion repeats: generate critical pairs, **select** one, add the rule.

Pending pairs grow **quadratically** in current rules — selection is the key choice point.

Selection Strategies: which inference should be selected next?

- TWEE picks the pair of least *weight* (symbol count).
- Classical strategies: Prioritize inferences matching **human-supplied** term patterns (e.g. resonators¹). Twitch originally motivated by desire to automate this idea.

¹L. Wos, *The resonance strategy*, Computers & Mathematics with Applications, 1995.

Motivating Example: Repeated Structure in Proofs

Fragment of a TWEE proof of LAT077-1, a lattice problem stated with only the Sheffer stroke f (NAND — it can express every Boolean operation; e.g. $f(t, t) = \neg t$):

$$\begin{aligned} & \vdots \\ &= f(x, f(y, f(f(z, f(x, x)), f(z, f(x, x)))))) \\ &= f(x, f(y, f(f(z, f(x, x)), f(f(x, x), z)))) \\ &= f(x, f(y, f(f(f(x, x), z), f(f(x, x), z)))) \\ &= f(f(f(x, x), f(x, x)), f(y, f(f(f(x, x), z), f(f(x, x), z)))) \\ & \vdots \end{aligned}$$

Motivating Example: Repeated Structure in Proofs

Fragment of a TWEE proof of LAT077-1, a lattice problem stated with only the **Sheffer stroke** f (NAND — it can express every Boolean operation; e.g. $f(t, t) = \neg t$):

$$\begin{aligned} & \vdots \\ &= f(x, f(y, f(f(z, f(x, x)), f(z, f(x, x)))))) \\ &= f(x, f(y, f(f(z, f(x, x)), f(f(x, x), z)))) \\ &= f(x, f(y, f(f(f(x, x), z), f(f(x, x), z)))) \\ &= f(f(f(x, x), f(x, x)), f(y, f(f(f(x, x), z), f(f(x, x), z)))) \\ & \vdots \end{aligned}$$

Terms of the form $f(t, t)$ — i.e. $\neg t$ — occur again and again. **Hypothesis:** the same shape is likely common in proofs of **related** problems.

Motivating Example: Repeated Structure in Proofs

Fragment of a TWEE proof of LAT077-1, a lattice problem stated with only the **Sheffer stroke** f (NAND — it can express every Boolean operation; e.g. $f(t, t) = \neg t$):

$$\begin{aligned} & \vdots \\ &= f(x, f(y, f(f(z, f(x, x)), f(z, f(x, x)))))) \\ &= f(x, f(y, f(f(z, f(x, x)), f(f(x, x), z)))) \\ &= f(x, f(y, f(f(f(x, x), z), f(f(x, x), z)))) \\ &= f(f(f(x, x), f(x, x)), f(y, f(f(f(x, x), z), f(f(x, x), z)))) \\ & \vdots \end{aligned}$$

Terms of the form $f(t, t)$ — i.e. $\neg t$ — occur again and again. **Hypothesis:** the same shape is likely common in proofs of **related** problems.

How do we **find** the patterns? How can the prover **exploit** them?

TWEE + STITCH = TWITCH.

Compression Turns Repeated Structure into an Abstraction

Q1: finding patterns Q2: exploiting them

Idea: formalise “repeated term structure” as a **compression** objective; the maximally compressive definitions are our **abstractions**.

Abbreviate a recurring pattern

$$\begin{array}{c} g(x) := f(x, x) \\ f(x, f(y, f(f(z, f(x, x)), f(z, f(x, x))))) \\ \Downarrow \\ f(x, f(y, g(f(z, g(x))))) \end{array}$$

STITCH searches for definitions minimizing:

$$\text{size}(A) + \sum_{t \in T} \text{size}(\text{rewrite}(t, A)).$$

How Should the Prover Use a Pattern?

Q1: finding patterns

Q2: exploiting them

1. Add a definitional axiom

$$\forall x. g(x) = f(x, x)$$

A **known technique** (TPTP's R0B033-1;
TWEE's **goal flattening**).

On LAT077-1 with $f(x, x)$: baseline ~ 250 s | as axiom ~ 10 s |

How Should the Prover Use a Pattern?

Q1: finding patterns

Q2: exploiting them

1. Add a definitional axiom

$$\forall x. g(x) = f(x, x)$$

A **known technique** (TPTP's R0B033-1;
TWEE's **goal flattening**).

But each axiom permits new inferences $\Rightarrow$
larger search space

On LAT077-1 with $f(x, x)$: baseline ~ 250 s | as axiom ~ 10 s |

How Should the Prover Use a Pattern?

Q1: finding patterns

Q2: exploiting them

1. Add a definitional axiom

$$\forall x. g(x) = f(x, x)$$

A **known technique** (TPTP's R0B033-1; TWEE's **goal flattening**).

But each axiom permits new inferences $\Rightarrow$ **larger search space**

2. Weighting (our approach)

$$\begin{aligned} w(f(x, f(y, f(f(z, f(x, x)), f(z, f(x, x)))))) \\ = 15 \end{aligned}$$

$$\begin{aligned} w(f(x, f(y, g(f(z, g(x)))))) \\ = 9 \end{aligned}$$

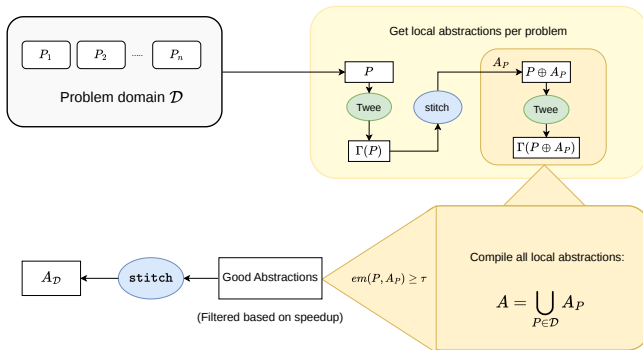
Change inference **selection order**. Weaker per pattern, but we can supply many.

On LAT077-1 with $f(x, x)$: baseline ~ 250 s | as axiom ~ 10 s | weighting ~ 130 s

Mode 1: Domain Abstractions

Mode 1: domain proofs

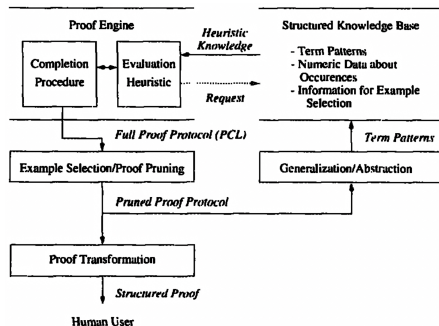
Mode 2: partial run on P



Learn local abstractions from each solved problem's proof; keep sets with speedup; pool the survivors and compress again $\rightarrow A_{\mathcal{D}}$

Related Work: Term Space Maps (Schulz et. al.)

DISCOUNT could use common structure from successful proofs to influence selection heuristic² — the same loop as our domain abstractions.



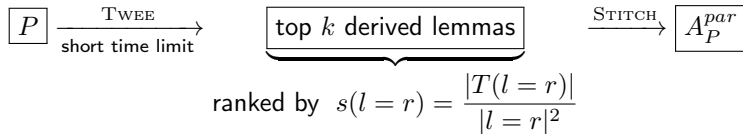
Differs in details of **patterns**: a term space map scores **how typical a term looks**, symbol by symbol; our abstractions ask whether it **contains** one of a explicit shapes derived via compression.

²S. Schulz and F. Brandt, *Using Term Space Maps to Capture Search Control Knowledge in Equational Theorem Proving*, FLAIRS-12, 1999; figure reproduced from that paper. See also Schulz, *Learning Search Control Knowledge for Equational Deduction*, 2000.

Mode 2: Partial Proof Abstractions

Mode 1: domain proofs

Mode 2: partial run on P



Heuristic scoring intuition

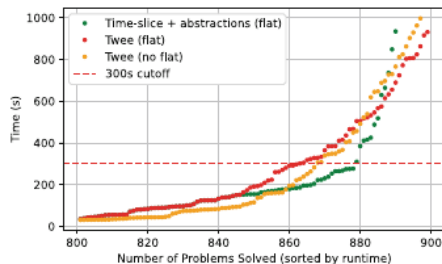
Prefer a lemma with a **simple statement** but a **long derivation**.

No related 'training' problems are required ³.

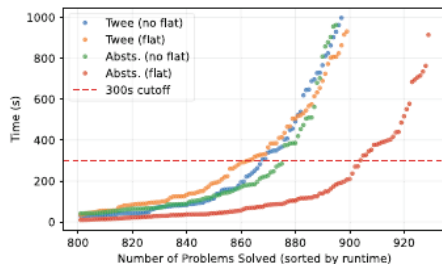
³Michael Rawson, *Heuristic Definition Introduction*, VAMPIRE23: THE 7TH VAMPIRE WORKSHOP, 2023.

Domain Abstractions Roughly Halve Runtime

TPTP 9.2.1: 1,041 UNSAT UEQ problems, 9 domains | each point one problem, sorted by runtime: **right** = more solved, **lower** = faster.



(a) Using partial proof abstractions.



(b) Using domain abstractions.

With goal flattening, domain abstractions solve about **25 more problems within 300 s**; on baseline-solvable problems, runtime is **roughly halved**. Partial-proof abstractions help, but less.

Abstractions Help Solve Hard Problems

70 hard problems (TPTP rating ≥ 0.9 and baseline T_{WEE} fails within 1,000 s)

11

partial proof abstractions

18

domain abstractions

Every problem solved using partial proof abstractions is **also** solved using domain abstractions.
But partial proofs need **no related problems at all**.

12 of the solved problems are rated **1.00**: unsolved by every ATP in TPTP's 300 s rating runs.

Domain: LAT with Sheffer stroke only

Domain Abstractions

$f(A, A)$	negation
$f(f(B, A), f(B, A))$	conjunction
$f(f(A, B), f(A, C))$	self-distributive shape

Effect on LAT075-1 (Rating: 1.00)

Baseline TWEE: Timeout (1,000 s)

With abstractions: 32 s

Manual ablation found the first and third patterns crucial.

Related Work: Compressing Proofs for *Concepts*

Vyskočil, Stanovský & Urban⁴ compress ATP proofs by **inventing definitions**: choose D minimising $w(D) + w(\text{proof rewritten by } D)$ — **the objective we hand to STITCH**. Their target is **humanization**, i.e. recovery of semantically meaningful definitions from long Prover9 proofs.

Case study on AIM proofs ⁵: "*many times the introduced definitions seem to be of little mathematical value.*" Similar result with STITCH.

⁴J. Vyskočil et al., *Automated Proof Compression by Invention of New Definitions*.

⁵J.D. Phillips, Stanovský, *Automated theorem proving in loop theory*.

Related Work: Compressing Proofs for *Concepts*

Vyskočil, Stanovský & Urban⁴ compress ATP proofs by **inventing definitions**: choose D minimising $w(D) + w(\text{proof rewritten by } D)$ — **the objective we hand to STITCH**. Their target is **humanization**, i.e. recovery of semantically meaningful definitions from long Prover9 proofs.

Case study on AIM proofs⁵: "*many times the introduced definitions seem to be of little mathematical value.*" Similar result with STITCH.

To try: Compression of different objects?

- **Cut-introduction** (Hetzl et al.)^a : compresses a proof's *Herbrand term set* — **relevant term instances** — producing a **quantified lemma** rather than a term pattern. Recovers *partial order* from axiomatic lattice proofs.
- Wernard, Zombori^b: compression of **proof structure**. Considers Condensed Detachment proof-terms. Can these ideas be generalized to other calculi (like completion).

^aG. Ebner et al., *On the Generation of Quantified Lemmas*.

^bC. Wernard, Z. Zombori., *Mathematical Knowledge Bases as Grammar-Compressed Proof Terms: Exploring Metamath Proof Structures*

⁴J. Vyskočil et al., *Automated Proof Compression by Invention of New Definitions*.

⁵J.D. Phillips, Stanovský, *Automated theorem proving in loop theory*.

Summary and Further Avenues

- Recurring (sometimes **interpretable**) term patterns can be extracted from proofs by **compression**, and applied through a weighting heuristic — biasing inference selection without changing the search space.
- Domain abstractions: runtime roughly halved, **18/70** hard problems solved. Partial-proof abstractions: weaker, but need no related problems.

Possible Future work

- **Domain construction**: curate or *generate* related problems (cf. Veroff's proof sketches).
- **Hints, cf Veroff**: also learn key *large* rules that a size-based heuristic would never select.
- **Demodulation-stable matching**: close abstraction patterns under the rewrite rules so weighting still applies after demodulation.
- **Beyond runtime**: select abstractions for proof length or cross-strategy robustness; neurally learn to generate them from (problem, abstraction) data.

Code and data: github.com/WeAreDevo/ijcar26-twee_abstractions

Questions?